

项目学习 · 第 5 章

第5章. LangGraph：多轮澄清、改选与 Web 主流程

怎样把太多候选变成用户能选择的少量资料？

学习目标

1. 用 State、Node、Edge 解释 Web 主流程。
2. 理解真实 Facet、候选收敛与最终输出规则。
3. 掌握 interrupt、checkpoint 与选择恢复。
4. 追踪同轮改选、跨轮改选、返回和前后端契约。

阅读方法：先看业务问题和图，再读名词与源码，最后复制完整实验验证。代码节选用于定位实现；完整实验会明确运行条件。

代码基准：2026-10-03 本地项目当前源码。默认配置与环境覆盖值可能不同；本文不以旧实施计划代替真实实现。

运行约定：命令从项目根目录执行。学习实验代码直接收录在 PDF 内；自行新建练习文件后复制运行，不另附脚本。服务实验的结果应由你实际运行记录。

学习产出：每章至少留下一份自己的输入、输出和解释。涉及检索质量、耗时或简历中的改善幅度，只采用有证据的数据。

本章目录

5.1 需求分析：为什么检索之后还要继续对话 3

5.2 核心概念：State、Node、Edge 与 Checkpoint 3

5.3 工作流全貌：模型节点与业务节点怎样连接 5

5.4 Facet：用真实候选生成可解释的选择题 7

5.5 选择与返回：业务快照怎样更新 9

5.6 interrupt 与 Command：暂停不是保存 Python 调用栈 10

5.7 会话与接口：前后端怎样交换状态 12

5.8 离线实验：真正观察状态、选择与重放 13

5.9 Studio、Trace 与问题定位 15

5.10 复习题、答案与源码阅读索引 16

学习目标：把当前 Web 检索流程拆成 State、Node 和 Edge；理解真实候选分组、暂停选择、历史快照与恢复改选；能够区分浏览器聊天记录与服务端 checkpoint。讲解以当前 workflow.py、service.py 和节点实现为准。

5.1 需求分析：为什么检索之后还要继续对话

5.1.1 一条排序列表没有解决所有问题

用户说“找三一电路图”，资料库可能返回不同机械型号、不同模块和不同资料类型。把前五条直接展示，虽然数量看起来简洁，却可能让用户误以为其他候选不存在；把三十条全部展示，又会增加阅读负担。

这个项目选择先检索较大的候选集合，再根据真实候选生成选择题。用户点击资料类型或具体分类后，程序缩小候选集合；默认不超过五条时才给出结果卡片。选择题不是大模型凭空生成的建议，而是对应候选 ID 的实际分组。

用户还可能选错。真实产品需要支持同轮改选、回到前一轮分类、甚至已经看到结果后再点击历史选项。要做到这些，系统必须保留当时的候选状态与执行位置，并处理框架恢复时的重放行为。

5.1.2 把需求拆成六项可验证规则

第一，普通问候可以直接回复。第二，检索问题经过理解、改写和召回。第三，零结果给出明确提示。第四，默认一至五条结果进入最终展示。第五，超过五条继续澄清。第六，选择与改选只允许对应真实候选，不接受任意字符串当作分类。

这些规则包含模型理解，也包含确定性业务判断。适合把工作划分成节点，把数据放入共享状态，再用边连接节点。这样阅读者可以分别观察“模型理解了什么”“数据库召回了什么”“程序根据什么条件继续”。

5.1.3 从真实 Web 入口往下读

当前 /api/chat 位于 app/api/chat.py，调用 run_search_workflow(sessionId, message)；/api/select 调用 resume_search_workflow(sessionId, optionValue)。二者都进入 app/graph/service.py。

service 负责把外部会话标识转换成 Graph 配置、初始化或恢复执行，以及把 Graph 结果转换为接口对象。workflow 负责搭建节点和边。nodes 目录负责具体计算。先分清这三层，再进入 LangGraph API，会更容易定位问题。

5.2 核心概念：State、Node、Edge 与 Checkpoint

5.2.1 State：跨节点流转的数据

State (状态) 是各节点共享的数据结构。当前项目使用 CircuitSearchState(TypedDict, total=False)，字段包括原问题、意图、候选列表、分类选项、已选条件、候选历史和最终结果。

TypedDict 帮助声明字典有哪些键及其类型，不等于每次节点执行都会像 Pydantic 一样完成完整运行时输入校验。total=False 表示这些字段在类型层面可以缺省；节点访问某字段前仍应考虑它是否已经写入。

源码片段，来自 `app/graph/state.py` 的部分字段：

代码块 python

```
1 class CircuitSearchState(TypedDict, total=False):
2     original_query: str
3     intent: dict[str, Any]
4     rewritten_query: str
5     candidate_ids: list[int]
6     candidate_documents: list[dict[str, Any]]
7     current_options: list[dict[str, Any]]
8     option_groups: dict[str, list[int]]
9     candidate_history: list[dict[str, Any]]
10    final_results: list[dict[str, Any]]
11    status: str
```

这只是源码节选，完整结构还包含过滤条件、已用 Facet 与当前提示等字段。节点通常返回需要更新的部分字段，框架把更新合并进状态。当前字段没有自定义 reducer，更新同名字段时采用覆盖方式，不会自动拼接列表。

5.2.2 Node：职责清晰的处理函数

Node（节点） 从 State 读取输入，完成一件工作，再返回状态更新。例如 `understand_query` 提取结构化意图，`retrieve` 生成候选，`build_facets` 生成分类选项。

一个节点可以调用模型、数据库或纯函数，LangGraph 不规定所有节点都必须调用 LLM。当前 `final_answer` 直接组织数量提示与真实候选，`clarify` 接受选择并过滤，都是理解“模型能力与程序规则分工”的好例子。

5.2.3 Edge：决定下一个节点是谁

Edge（边） 控制执行路线。固定边总是通往指定节点；**Conditional Edge（条件边）** 先执行路由函数，再根据返回标签选择下一步。START 与 END 是框架提供的起点、终点标记。

当前 `rewrite_query` 后固定进入 `retrieve`。`retrieve` 后则根据候选数量进入无结果、最终展示或澄清。用户选择之后再次按数量路由，因此形成“分组—选择—缩小—再次判断”的循环。

5.2.4 Checkpoint 与 Thread

Checkpoint（检查点） 记录图执行中的状态与执行信息，支持暂停和恢复。**Checkpointers（检查点存储器）** 负责保存与读取这些信息。**Thread（执行会话）** 用一个 `thread_id` 把同一会话的检查点关联起来。

业务层的 `candidate_history` 也是历史，但它是项目自己保存的候选快照。Checkpoint 由框架管理，记录执行进度；`candidate_history` 由业务函数管理，用于“返回上一步”。两者用途不同，下一节会具体区分。

对象	保存什么	当前实现中的用途
State	当前问题、候选、分类与结果	节点之间交换数据
candidate_history	选择前的候选及分类状态	返回上一步
Checkpoint	框架状态与执行信息	中断恢复、历史分支重建
thread_id	检查点会话键	区分不同浏览器会话
localStorage	前端保存的消息与卡片数据	恢复界面历史

5.3 工作流全貌：模型节点与业务节点怎样连接

5.3.1 先观察完整路线

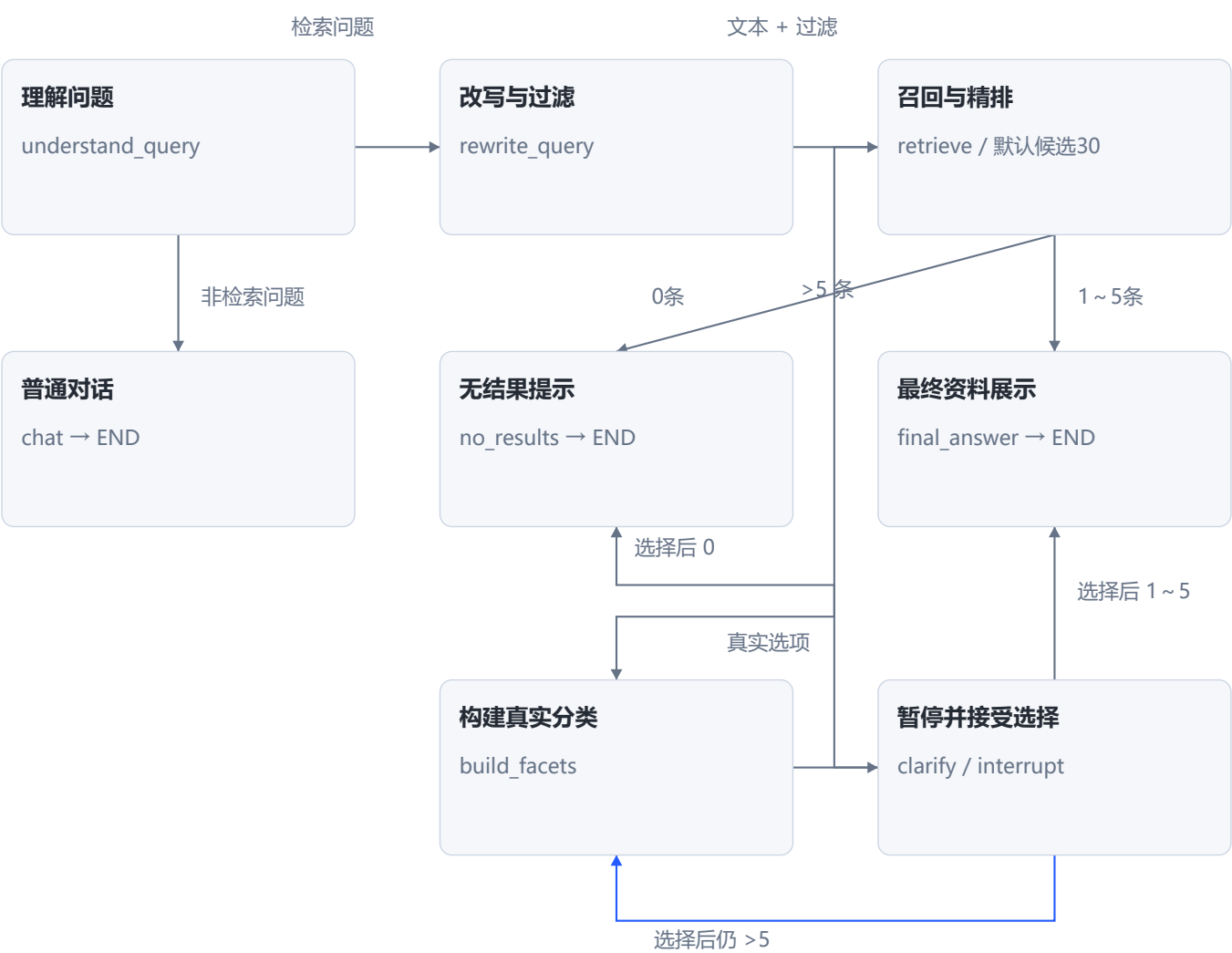


图5-1. 理解节点分流普通对话和检索；候选数为0、1~5、超过5分别进入无结果、最终展示和澄清。选择后再次按数量判断，阈值可配置。

理解节点决定普通对话或检索；候选数量控制最终展示与澄清；澄清后重复执行数量判断，不需要每轮重新访问数据库。

app/graph/workflow.py 注册八个节点：understand_query、chat、rewrite_query、retrieve、build_facets、clarify、final_answer、no_results。图编译后才能执行，构建阶段只是注册函数与路径。

普通问候走 understand_query → chat → END；检索问题走 understand_query → rewrite_query → retrieve。检索结果多时进入 build_facets → clarify，选择完成后再次检查候选数量。

5.3.2 理解与改写承担什么工作

understand_query 调用 parse_search_intent()，把 Pydantic 输出变成字典写入 State。route_query 查看 is_search_request，为真时进入改写，否则进入普通对话。

当前路由在该字段缺省时使用 True，倾向进入检索。不要把这个默认值理解成模型永远正确识别查询；真正的识别质量要通过意图数据与模型实验评价。

rewrite_query_node 把 intent 字典重新校验为 SearchIntent；对原问题做检索改写；根据结构化意图构建元数据过滤表达式。改写文本与过滤表达式是两个输出，作用分别是改善文本召回与缩小检索范围。

5.3.3 候选 30 条与最终 5 条为什么不同

当前配置默认 graph_candidate_k=30、graph_max_results=5、graph_max_options=5。候选数量是召回与精排阶段保留的规模；最终结果上限是业务展示阈值；选项上限是单轮交互负担的限制。

retrieve 对混合召回和 Cross-Encoder 都传入候选规模 30，暂时保留足够资料供澄清。数据不足或评分阈值淘汰候选时，实际数量会小于 30。后续用户选择也会持续缩小集合。

默认值	控制什么	为什么不能混淆
graph_candidate_k = 30	初始候选规模	给澄清保留足够选择空间
graph_max_results = 5	最终展示阈值	默认超过五条要继续收敛
graph_max_options = 5	一轮选项总数	返回按钮也要占一个名额

这些值来自配置，可被环境配置改变。文档中的“一至五条”描述当前默认规则；源码实际比较的是 settings.graph_max_results，并非在所有位置都写死数字 5。

5.3.4 过滤失败后的退路

当过滤条件存在且精排后候选为空，retrieve 会再做一次不带过滤条件的混合召回与精排。判断发生在精排之后，因为有过滤召回结果也可能全部被评分阈值淘汰。

退回无过滤检索后，状态设置 filter_fallback=True，并清空当前 filter_query。这表示此次已放宽硬过滤，应在调试时重点观察候选是否相关。放宽过滤不是凭空增加资料，也不代表一定能找到结果。

5.3.5 三路数量规则如何落实

源码片段，来自 `app/graph/nodes/retrieve.py`：

代码块 python

```
1 count = len(state.get("candidate_documents", []))
2 if count == 0:
3     return "no_results"
4 if count <= settings.graph_max_results:
5     return "final_answer"
6 return "build_facets"
```

这个路由函数同时接在 `retrieve` 与 `clarify` 后面。因此初始三条直接展示；初始八条继续分组；选择后剩两条再展示；极端情况下选择后零条进入无结果。

`final_answer` 还有一次切片限制 `[:settings.graph_max_results]`，提供最终输出的数量保护。正文只包含一句数量与排序提示，具体条目由前端卡片渲染；没有再让模型生成一组新的文档标题。

5.4 Facet：用真实候选生成可解释的选择题

5.4.1 Facet 的含义

Facet (分类维度) 是把同一批候选按某种属性拆成若干组，例如资料类型、层级路径中的某一段。当前实现读取标题与层级路径，选项必须映射到这批候选的实际文档 ID。

`option_groups` 保存 选项值 → 文档 ID 列表；`current_options` 保存前端需要的 `id`、`text`、`value`。显示文字面向用户，`value` 面向程序，真正决定过滤的是映射中的 ID 集合。

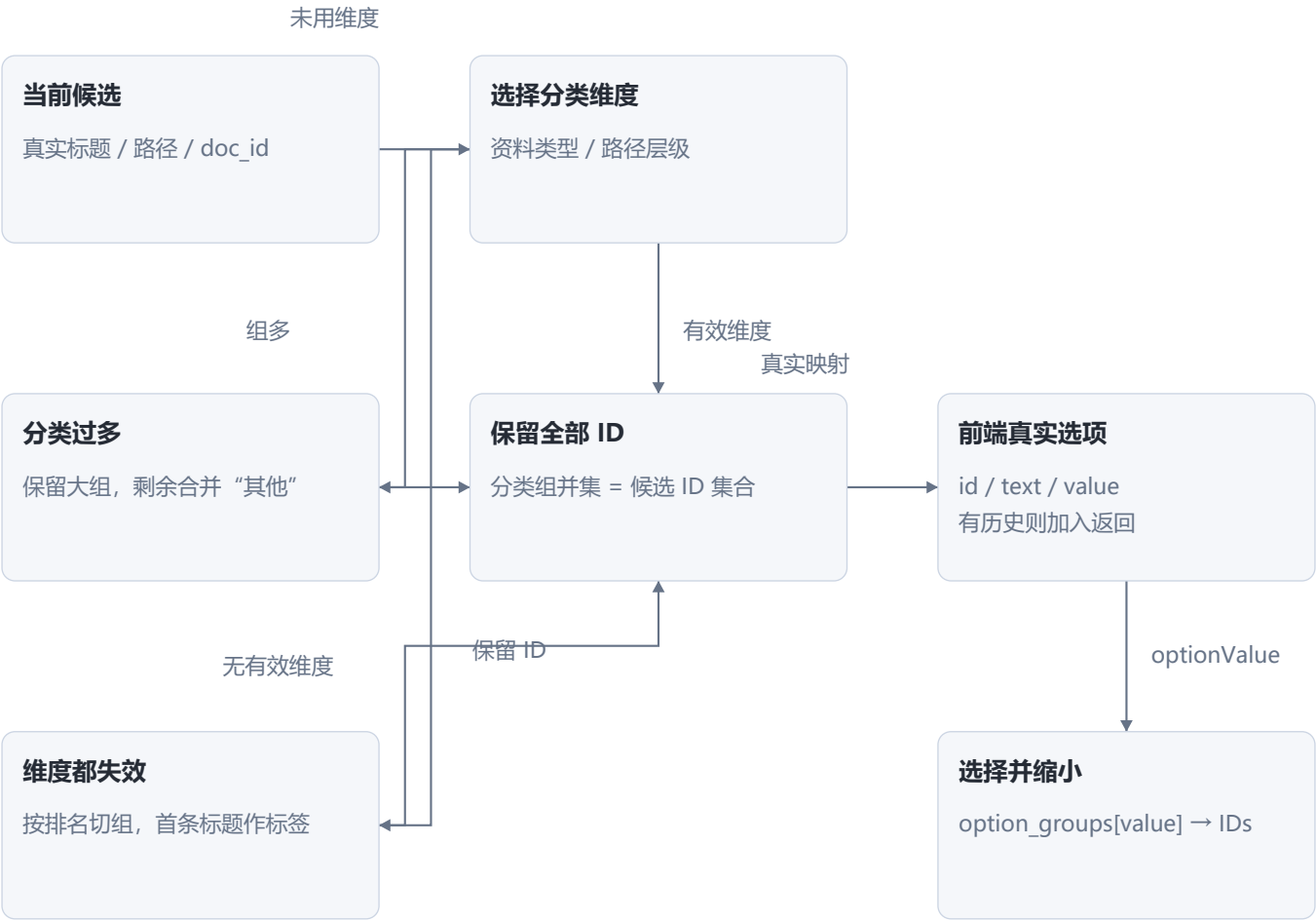


图5-2. 分类选项映射到真实候选 ID；自然有效的分类可为2~5个，有历史时预留返回位。分组过多合并余桶，维度失效则按排名兜底。

每个选项都携带候选 ID；分类过多时合并余桶，避免丢失资料；有历史时预留“返回上一步”的选项位。

5.4.2 有哪些可用维度

第一类是资料类型。程序从标题和路径中寻找“整车电路图”“电路图”“针脚定义”“维修手册”等关键词；关键词采用较长词优先的顺序，减少“整车电路图”过早被“电路图”截取的情况。

第二类是路径层级。路径先按 -> 拆段，第 0 层显示“资料大类”，第 1 层显示“资料类型”，其他层使用“更具体的分类”。不同资料分支的层级语义并不统一，所以程序没有把每个第三、第四层都武断叫作品牌或车型。

某条资料没有对应层级或没有识别出的资料类型时，会进入“未分类”。这可以保留候选完整性，避免解析不足就悄悄丢弃记录。学习资料数据时，缺失值的处理同样属于业务行为。

5.4.3 维度选择不是固定先问品牌

程序先考虑自然形成 2 到可用选项数的维度，优先选分组数更多者；分组数相同时优先资料类型。这样通常能让一轮选择产生更明显的候选区分。

如果所有有效维度都分组过多，再考虑合并后的维度，优先使合并后最大一组尽可能小。否则一个巨大的“其他”组会让用户点击后仍面对大量候选。测试 test_select_dimension_prefers_smaller_other_bucket 专门覆盖这一点。

这里所谓“2 到 5 个分类”允许两组。clarify 函数的文档字符串写了“3~5”，但实际分组函数允许自然产生两个分类；应以代码与测试为准。若存在返回按钮，可用于分类的名额会从 5 降到 4。

5.4.4 其他余桶如何保证覆盖

分类过多时，程序按组规模从大到小排序，保留前 `max_options-1` 组，把余下组的 ID 合并到“其他”。“其他”是一组实际候选，不是模型临时发明的类别。

例如有八个品牌且本轮最多五个选项，保留四个较大的品牌组，剩余品牌一起进入“其他”。若本轮需要返回按钮，分类名额只有四个，就保留三个组并把剩余组并入“其他”，最后加返回按钮。

合并后应该满足一个重要不变量：所有分类组的 ID 并集等于当前候选 ID 集合。覆盖测试可以证明这条规则。分类标签的可读性与候选覆盖正确性都需要考虑，不能只检查按钮数量好看。

5.4.5 所有语义维度都失效时怎么办

当资料类型与路径都无法拆开候选时，程序按当前排序切组，组标签取组内首条真实标题的短名称。它不会调用模型为候选编造一个新型号。

这种按排名分组是交互兜底，不代表组内具有共同业务属性。学习时要明确说明：“这一组以首条标题帮助辨认，组内资料按排名相邻。”如果希望分类更有业务意义，应改进数据字段或分类规则。

兜底 Facet 键带轮次后缀，例如 `rank_group_0`。这样下一轮仍能继续按排名收敛，不会被已使用维度列表误判为不能重复处理。

5.5 选择与返回：业务快照怎样更新

5.5.1 apply_selection 的四步处理

`app/graph/nodes/clarify.py` 的 `apply_selection()` 首先根据 `option_value` 读取 ID 列表，拒绝不存在或为空的选项；然后从当前候选中筛出这些 ID 对应的记录。

接着把选择前的候选、ID、已选条件、已用 Facet 深拷贝到 `candidate_history`。最后记录本轮 Facet 和选择文字，递增澄清轮次，清空已经消费的选项字段，返回新的状态更新。

深拷贝用于避免后来修改列表或字典时把旧快照一并改变。候选历史保存的是回退所需的业务数据，没有替代框架 checkpoint。框架历史负责重新进入某个执行点；业务历史负责恢复候选集合。

5.5.2 used_facets 与 selected_filters 的作用

`used_facets` 记录已经问过的维度，下一轮构建选项时跳过这些维度，减少对同一分类反复提问。`selected_filters` 记录维度及用户看到的选项文字，便于解释已选条件。

选中分类并没有再次执行 Milvus 过滤查询。此阶段是对已检索候选做 ID 集合过滤。不要把 `selected_filters` 这个字段名称自动理解成数据库表达式；真正传给召回器的过滤表达式是前面的 `filter_query`。

5.5.3 返回上一步怎样恢复

保留选项值 `_back_` 表示返回上一步。程序从 `candidate_history` 弹出最近快照，恢复选择前的候选、已选条件和已用维度，并减少轮次；随后清空当前选项，让后续路由重新生成合适的分类。

没有历史时点击返回会抛出明确错误。历史存在时，返回按钮也必须计入选项上限，不能在已经有五个分类的情况下再无条件增加第六个按钮。

业务回退与点击旧选项有区别。返回从当前历史栈退一层；点击旧选项由 service 找到对应历史 checkpoint，再重新生成那轮中断。两条路径都服务于修改选择，但处理依据不同。

5.6 interrupt 与 Command: 暂停不是保存 Python 调用栈

5.6.1 interrupt 将什么交给前端

源码片段，来自 clarify 节点：

```
代码块 python
1 selection = interrupt({
2     "type": "options",
3     "content": state["current_prompt"],
4     "options": state["current_options"],
5 })
```

interrupt (中断) 让图暂停，携带需要交给用户的选择题。执行结果中的 `__interrupt__` 被 service 提取，转换为 `GraphResponse` 的 `options` 类型。服务端 `checkpointer` 保存相关状态。

用户点击后，服务端使用 `Command(resume=option_value)` 提供恢复值。在被恢复的节点中，这个值成为对应 `interrupt` 调用的返回值，然后进入 `apply_selection`。

5.6.2 为什么不能说“从原代码下一行继续”

恢复涉及节点重新执行与中断值重放，而不是保存一段正在运行的 Python 调用栈。节点里位于 `interrupt` 前面的逻辑可能再次执行。教学和调试时必须理解这一点，否则会把重复输出、重复副作用误判成框架异常。

当前 `clarify` 在 `interrupt` 前只是构造选择题数据，没有重新检索，也没有外部写操作，因此重建这一步相对安全。如果未来在中断前加入订单写入、邮件发送等操作，需要额外设计幂等性和职责拆分。

5.6.3 当前项目为什么要先重建中断

简单示例通常直接 `resume`。当前项目还需要支持已完成后的改选，以及一次非法值失败之后继续选择，因此增加了 `_prepare_resume()`。

历史 `checkpoint` 可能已经写入过旧恢复值。直接向旧执行点再次发送新值，可能重放旧值；一次非法值还可能留在失败执行的 `checkpoint` 中，后续合法值也被旧值影响。因此项目统一寻找能接受新选项的中断，再从其父 `checkpoint` 重新生成一个干净中断。

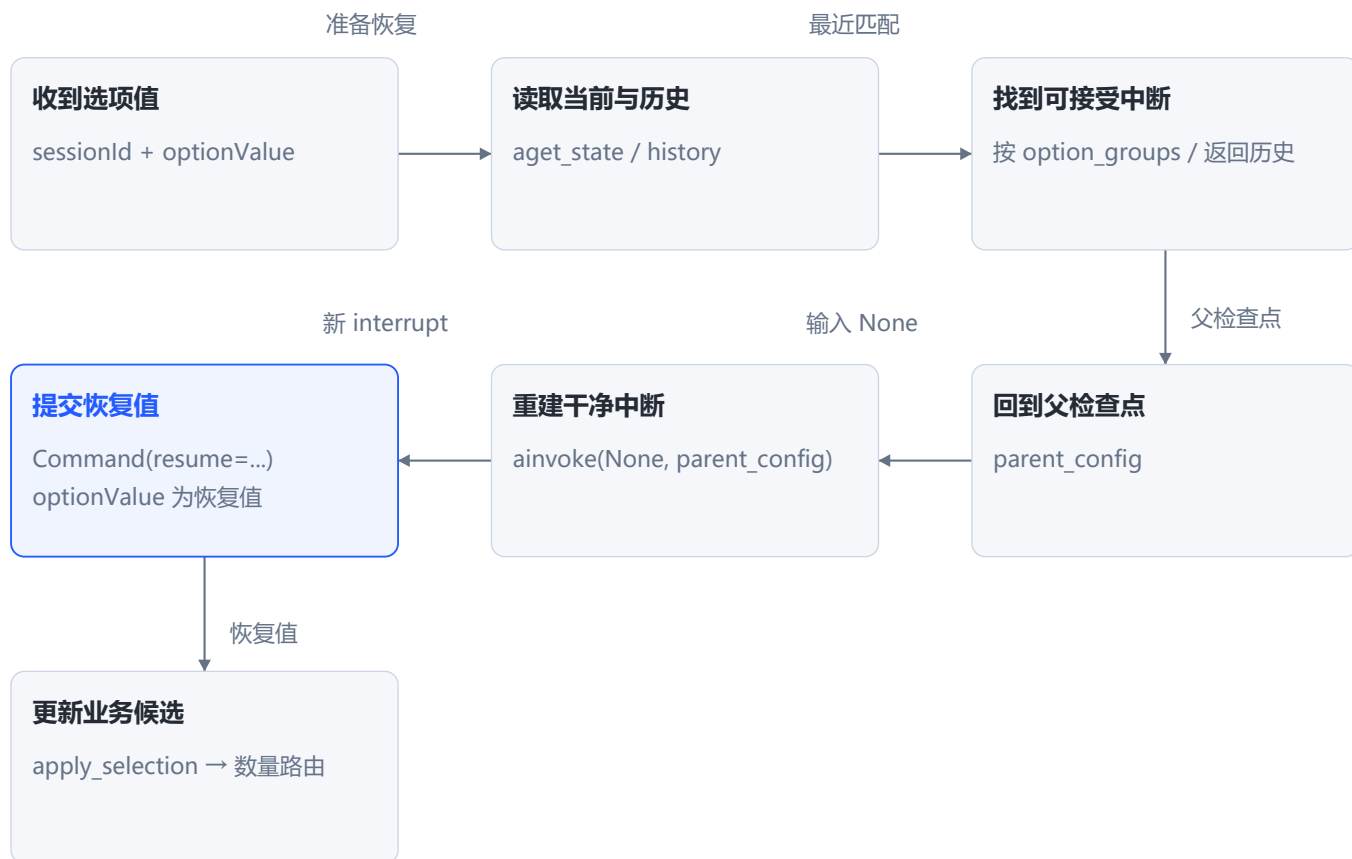


图5-3. 首次合法选择和改选都先寻找可接受值的中断，从 parent_config 重建干净中断，再提交 Command(resume=...)。没有状态时要求重新查询。

按选项值寻找可接受的历史中断，回到父 checkpoint 重建，再提交新 resume 值；首次合法选择也走相同准备流程。

5.6.4 _prepare_resume 的准确阅读顺序

第一，读取当前 snapshot，检查当前或任务内是否存在待处理中断。第二，查看当前 option_groups 是否包含用户选项，返回键则要求存在 candidate_history。第三，遍历历史 snapshot，筛出仍带中断且接受该值的记录。

第四，从候选记录中找到可用 parent_config，执行 await graph.ainvoke(None, config=parent_config)。这里输入 None，目的是从父检查点重算澄清节点，生成新中断；随后调用方才执行 Command(resume=option_value)。

当前实现的首次合法点击、同轮改选、跨轮改选都遵循这一流程。resume_search_workflow 上方还有“首次选择直接用”的旧注释，但函数实现和测试明确证明首次合法点击也会重建，应以代码行为为准。

5.6.5 非法值、无可可用状态与历史歧义

如果当前有待处理中断，但该值不属于任何可匹配选项，准备函数不重建，继续交给节点抛出“无效的选择项”。如果当前没有中断，也找不到可重建的历史执行点，则提示重新输入查询，可能原因是会话已不可恢复或服务器重启。

历史选择按 option_value 匹配最近记录。当前值形如 path_level_3:1，没有携带独立的查询编号或全局唯一选项令牌。因此同一 thread 的不同历史轮次出现相同值时，可能匹配较近的记录，而非用户肉眼所点的那张很旧的选

择题。

这是一项当前能力边界：文档可以说明支持按历史值回溯改选，但不能宣称任意旧消息都能精准定位原始轮次。新查询会显式重置业务候选状态，框架历史仍需结合当前实现理解；未来可研究把查询标识与轮次令牌加入选项契约。

5.7 会话与接口：前后端怎样交换状态

5.7.1 sessionId 怎样成为 thread_id

`_config(session_id)` 返回 `{"configurable": {"thread_id": session_id}}`。新问题与选择请求必须使用相同会话标识，否则 `checkpointer` 无法找到对应状态。

`run_search_workflow()` 初始化候选、历史、轮次、选项和结果等字段，避免新问题沿用上一轮业务状态。保留同一 `thread_id` 是会话关联方式，不代表新问题自动继续旧候选收敛。

5.7.2 InMemorySaver 的实际持久性

FastAPI 路径的 `get_circuit_graph()` 使用 `InMemorySaver()`，并通过 `lru_cache` 复用同一图实例。这样同一服务进程内可以保留会话 `checkpoint`。

服务器重启后，这些内存状态不会像磁盘数据库一样保留。多进程或多实例部署也不能默认共享彼此的内存状态。当前项目展示了本地会话恢复机制，若需要长期、多实例恢复，需要单独设计持久化 `checkpointer` 与部署方案。

5.7.3 浏览器 localStorage 保存的是界面历史

前端 `frontend/js/chat.js` 使用 `chatbot_sessions` 保存消息、选项及结果卡片数据。切换历史会话时恢复 `sessionId`，重建界面，并允许历史按钮再次点击。

但浏览器保存的按钮不会自动恢复服务端内存 `checkpoint`。服务器重启后，界面仍可能显示旧选择题，点击时后端却没有可用状态。这正是必须把“聊天记录还在”和“工作流还能恢复”分开理解的原因。

5.7.4 optionId 与 optionValue 分别负责什么

选择请求包含 `sessionId`、`optionId`、`optionValue`。当前后端真正传给恢复函数的是 `optionValue`；`optionId` 在请求 `Schema` 中保留，但没有作为筛选候选的判断依据。

界面的 A、B、C 是显示字母；`option.id` 是按钮数据中的编号；`option.value` 才是后端业务映射键。调试时不要把这三个值混为一谈，否则容易出现“点了 B 但传了字母 B”这样的契约错误。

5.7.5 三类响应如何转换与渲染

type	服务端含义	前端呈现
text	普通对话或无结果提示	文本消息
options	Graph 正在等待选择	提示文字与选项按钮
result	已得到默认一至五条最终候选	数量提示与资料卡片

Graph 内部使用 doc_id、title、hierarchy_path；service 转换为前端 id、fileName、hierarchyPath，并补 keywords 与 score。API 同时保留 documents 列表和首条 document，帮助兼容旧前端读取方式。

前端卡片展示真实标题、ID、路径，匹配分存在且不为零时额外显示。标题注入使用 textContent，避免把资料名称当 HTML 执行。学习接口时要检查字段适配与渲染路径，而不只看后端返回了什么。

5.7.6 错误响应怎样区别

接口封装 ApiResult，成功 code 为 1，业务错误 code 为 0。/api/select 会把 ValueError 的具体提示返回给前端；其他异常返回选择处理失败。/api/chat 分别处理配置 RuntimeError 与一般 workflow 异常。

Schema 校验失败、HTTP 状态与业务 code 是不同层面的结果。看到网络请求返回，不代表业务成功；前端还会检查 code。排错时同时查看请求体、接口响应与服务端日志。

5.8 离线实验：真正观察状态、选择与重放

5.8.1 实验一：最小中断图

可独立复制运行。项目根目录使用已有虚拟环境执行，无需模型或服务：

先在当前 PowerShell 窗口执行以下两行，关闭离线实验的可选追踪；它们不会修改项目 .env：

代码块 powershell

```
1 $env:LANGSMITH_TRACING = "false"
2 $env:LANGCHAIN_TRACING_V2 = "false"
```

代码块 python

```
1 from typing import TypedDict
2 from langgraph.graph import StateGraph, START, END
3 from langgraph.checkpoint.memory import InMemorySaver
4 from langgraph.types import Command, interrupt
5
6 class State(TypedDict, total=False):
7     chosen: str
8
9 def choose(state):
10     print("before interrupt")
11     value = interrupt({"options": ["A", "B"]})
12     return {"chosen": value}
13
14 builder = StateGraph(State)
15 builder.add_node("choose", choose)
16 builder.add_edge(START, "choose")
17 builder.add_edge("choose", END)
18 graph = builder.compile(checkpointer=InMemorySaver())
19 config = {"configurable": {"thread_id": "offline-demo"}}
20 first = graph.invoke({}, config=config)
21 print("paused:", bool(first.get("__interrupt__")))
22 last = graph.invoke(Command(resume="B"), config=config)
23 print("chosen:", last["chosen"])
```

预期并实际观察到：首次打印 before interrupt, paused 为 True；恢复时再次打印 before interrupt, 最终 chosen 为 B。两次打印说明恢复会重新进入节点，不能解释成保存 Python 栈后直接执行下一行。

这个实验是最小教学图，没有实现项目的历史改选重建。它只证明中断、thread 配置和恢复值的基本关系，随后再阅读 _prepare_resume() 的额外业务需求。

5.8.2 实验二：真实 Facet 与候选回退

可独立复制运行。以下六条记录是教学构造，不是新增到真实 CSV 的资料：

代码块 python

```
1 from app.graph.nodes.facets import build_facets
2 from app.graph.nodes.clarify import apply_selection
3
4 docs = []
5 for index, brand in enumerate(
6     ["三一", "三一", "徐工", "徐工", "卡特", "卡特"], 1
7 ):
8     docs.append({
9         "doc_id": index, "title": f"{brand}_电路图",
10        "hierarchy_path": f"电路图->ECU->工程机械->{brand}"
11    })
12 state = {"candidate_documents": docs,
13         "candidate_ids": list(range(1, 7)),
14         "candidate_history": [], "used_facets": []}
15 state.update(build_facets(state))
16 print(state["current_facet"])
17 print([item["text"] for item in state["current_options"]])
18 value = state["current_options"][0]["value"]
19 state.update(apply_selection(state, value))
20 print("selected:", state["candidate_ids"])
21 state.update(apply_selection(state, "__back__"))
22 print("restored:", state["candidate_ids"])
```

实际 Facet 为 path_level_3，产生三一、卡特、徐工三个真实组，具体顺序受当前排序规则影响。选中首组剩两个 ID；返回后恢复 [1, 2, 3, 4, 5, 6]。

修改练习：把六条资料标题改成两种资料类型，再观察是否优先按 document_type 分类；把路径都改成相同值并去掉类型关键词，再观察 rank_group 兜底。练习目标是检查分类来源，不是只看按钮数量。

5.8.3 实验三：检查默认数量路由

可独立复制运行：

代码块 python

```
1 from app.graph.nodes.retrieve import route_by_result_count
2 from app.core.config import get_settings
3
4 limit = get_settings().graph_max_results
5 for count in [0, limit, limit + 1]:
6     result = route_by_result_count({
7         "candidate_documents": [{}] * count
8     })
9     print(count, result)
```

当前默认配置下，实际结果是 0 no_results、5 final_answer、6 build_facets。代码从 settings 读取阈值，因此自定义配置下输出数字可能不同，但分支关系应保持一致。

5.8.4 怎样验证改选恢复而不调用模型

本次实际执行了 tests/test_graph_resume.py。它使用 FakeSnapshot 与 FakeGraph，检查首次选择重建、结束后改选、跨轮回溯、非法值保留错误路径，以及缺少可恢复状态的行为。

这些测试直接断言调用记录为 (None, parent_config)，因此可以证明准备函数走了父 checkpoint 重建路径。它们使用假的图对象，不能单独证明真实 LangGraph 在所有异常序列中的集成行为；最小中断实验另外观察了真实框架的节点重放。

本章与上一章相关六个测试文件本次合计实际通过 24 项。实验没有重新调用付费模型、访问 Milvus 或启动 Web 服务。真实端到端检索、分类效果与恢复交互仍需要在依赖服务齐备后单独观察。

5.9 Studio、Trace 与问题定位

5.9.1 Studio 与 FastAPI 使用同一图定义

build_circuit_graph(checkpointer=None) 定义共享图结构。FastAPI 的 get_circuit_graph 传入 InMemorySaver；导出的 studio_graph = build_circuit_graph() 没有自己传入该内存存储器。

langgraph.json 指向 ./app/graph/workflow.py:studio_graph。Studio 的 Agent Server 运行时提供自己的 checkpointer。不能把 FastAPI 的内存会话与 Studio 中的 thread 自动视为同一份状态。

本次没有启动 Studio。学习时可以先画节点关系，再在环境配置完成后打开 Studio，检查真实节点输入输出是否与自己的理解一致。界面只是观察入口，状态语义仍需要回到代码解释。

5.9.2 Trace 适合回答哪些问题

Trace（执行追踪）可以帮助观察模型调用、节点耗时、查询改写、候选数量与错误发生位置。项目环境示例提供 LANGSMITH_TRACING，默认值为 false，另有项目名与 API Key 配置项。

当前文档不能据此宣称生产追踪已经启用。若未来启用，应观察模型输入输出与实际检索记录，并理解哪些数据会进入追踪系统。Trace 是调试证据，不能替代测试或完整的质量评估。

5.9.3 按现象逐层定位

现象	优先观察	常见解释
查询直接进入闲聊	intent 与 route_query	is_search_request 被判为 False
召回结果为零	filter_query、精排阈值、fallback	过滤或阈值可能淘汰候选
分类看似无意义	Facet 键与真实标题	可能进入按排名兜底
点击不同选项结果相同	_prepare_resume 调用与历史	旧 resume 重放或匹配轮次错误
历史按钮无法继续	thread_id 与进程状态	服务端内存可能已丢失
结果有数据但卡片空白	字段转换与前端渲染	snake_case 与 camelCase 契约问题

排查“点哪个都一样”时，不要第一反应重新调整大模型提示词。先检查选项是否映射到不同 ID，以及是否从父 checkpoint 重新生成了中断。排查卡片问题时，也不应立即重新召回；可能只是字段或浏览器 DOM 使用错误。

5.10 复习题、答案与源码阅读索引

5.10.1 复习题

- 1. 为什么 State 字段可以只返回部分更新？当前列表会自动追加吗？
- 2. 候选 30、最终 5、选项 5 分别控制什么？
- 3. 两个真实分类是否合法？有返回按钮时还能放五个分类吗？
- 4. “其他”余桶怎样保证候选不丢失？
- 5. candidate_history 与 checkpoint 分别负责什么？
- 6. 为什么 interrupt 恢复不能理解成原 Python 栈继续执行？
- 7. 当前首次合法选择为什么也调用 ainvoke(None, parent_config)？
- 8. 浏览器历史仍然存在时，为什么服务器重启后无法恢复选择？
- 9. optionId、界面字母和 optionValue 哪一个真正控制过滤？

5.10.2 参考答案

节点返回状态更新，框架合并同名键；当前字段无自定义 reducer，列表更新采用覆盖。30 保留候选空间，结果 5 控制展示，选项 5 控制一轮交互总数。两个分类合法；返回占一个名额，有返回时最多四个分类。

余桶合并剩余组的真实 ID，分类 ID 并集应覆盖所有候选。candidate_history 保存业务回退数据；checkpoint 保存框架执行状态。恢复会重新进入节点并重放中断值，因此中断前的代码可能再次执行。

首次合法选择统一重建，避免旧或失败恢复值污染后续选择路径。浏览器 localStorage 保存界面消息，服务器 InMemorySaver 保存进程内状态，两者不是同一持久化系统。后端实际按 optionValue 查映射，optionId 与字母是其他层面的标识。

5.10.3 源码阅读索引

阅读顺序	源码位置	读完应能解释
1	app/graph/state.py	哪些状态跨节点流转
2	app/graph/workflow.py	节点、固定边、条件边怎样连接
3	app/graph/nodes/retrieve.py	候选规模、过滤回退、数量路由
4	app/graph/nodes/facets.py	真实分类、余桶、排名兜底
5	app/graph/nodes/clarify.py	暂停、过滤、历史与返回
6	app/graph/service.py	thread 配置、重建中断、结果转换
7	app/api/chat.py	外部请求和错误怎样封装
8	frontend/js/chat.js	点击、历史与结果卡片如何呈现
9	tests/test_graph_resume.py	改选恢复的程序契约

用自己的话描述一次八条候选的查询：模型理解并改写需求，程序召回八条，按真实数据构建分类，Graph 暂停；用户提交选项，service 准备干净中断并恢复，程序按 ID 缩小候选；剩余数量决定结果展示或下一轮分类。再解释返回与点击旧选项的区别，以及当前进程内状态的限制。

进一步查阅官方文档：[Interrupts](#)、[Persistence](#)。重点对照节点重放与 thread/checkpoint 概念，避免只复制恢复调用而忽略实际状态行为。